

Asad Ikram

+44 7482 376417 · asad.ikram53@gmail.com · linkedin.com/in/asad-ikram98 · asad-ikram-mc.github.io/portfolio

Dr Rosa Sanchez Salgado, Dr Carolin Ischen and Prof. Gijs Schumacher
A.I. Feels project, Faculty of Social and Behavioural Sciences
University of Amsterdam

September 2026

Application: PhD position on emotion-sensitive A.I. in politics and policymaking (A.I. Feels, SP1), vacancy 15455

Dear Dr Sanchez Salgado, Dr Ischen and Professor Schumacher,

I have taught machines to read emotion in what people write, and now I want to study the emotion machines write for policymakers. For seven years, across more than five companies and over 300 freelance clients, I have mostly collected and analysed online data. Before ArtemisAI's classifiers learned from Claude and DeepSeek, we labelled about 500 posts and comments by hand and checked both models against them and against each other. The two disagreed about emotion more often than about sentiment, so I learned to treat a model's reading of emotion as an interpretation to be tested. SP1 asks the mirror question, and Dr Sanchez Salgado's work on feeling rules suggests where to look. I hold an MSc in Data Analytics (Distinction) from the University of Portsmouth and a BSc in Computer Science from FAST-NUCES, and am a Data Engineer at M+C Saatchi Fluency and founder and CTO of ArtemisAI. I am applying for the A.I. Feels PhD position in sub-project 1.

At Dubizzle Labs (Senior Data Engineer, 2022 to 2024, promoted twice) I led three engineers running more than 500 collectors in four languages across 15 countries, and analysed e-commerce and real estate stores and online discussions to build competitor-analysis dashboards that predicted revenue and showed what customers said. People wrote differently on forums and on social platforms, so anything measured from their words depended on where it was collected. My Chevening Scholarship (2024/25) rested on one argument: public decisions deserve better evidence about what people think. When that evidence reaches civil servants through chatbots, its wording becomes evidence too.

At Portsmouth my dissertation studied about 279,000 posts on UK economic policy from four platforms, with a fine-tuned BERT classifier (macro-F1 0.878), a 200-post human-coded validation set (Cohen's kappa about 0.78) and NRC counts of eight emotions. With Dr Ella Haig, my supervisor, I turned its finding that the same policy event looked structurally different on each platform into a methods manuscript that treats each platform as a survey mode and adapts measurement invariance to machine-labelled sentiment. In its 500-replication simulation, a naively pooled index gives the wrong sign on nearly a quarter of days. Writing it up, I found that my own 0.878 had been scored against lexicon-seeded labels rather than the human coders, and the manuscript says so.

At Fluency, where I have worked since December 2024, we analyse online data. I have supported social listening pipelines for UK Government departments and Nike and built others for Amazon and Ford. Where I can share methods but not findings, I built a codebook for a US women's soccer project from BERTopic topics I merged by hand and checked an LLM-tagging pilot by hand, and, for Abbott, screened about 170,000 multilingual posts and reviews in LLM-assisted stages and coded the relevant ones against a 24-emotion taxonomy. When a client questioned a CFMOTO launch report, we compared two tagging runs and reported as directional anything the checks did not support.

ArtemisAI is running in production ahead of our December launch, turning model outputs into advice a small business owner would act on without being able to check it, such as flop warnings and crisis alerts. Its six fine-tuned encoder classifiers, including a DistilBERT emotion model, code over three million comments. After our hand labels, the classifiers learned from Claude (about 6,000 posts) and DeepSeek (about 12,000 more). The two agreed on 82.9 per cent of sentiment labels but 74.4 per cent of emotion labels, and on no toxicity labels because their taxonomies were incompatible. From the fourth version on, the classifiers were scored against Claude's labels, so the emotion model's accuracy of 84 per cent partly measures agreement with another model, not with people.

Dr Sanchez Salgado's analysis of feeling rules in European policymaking (JCMS, 2023) shows emotions treated as inappropriate in health, energy and environment policy but acceptable in foreign and home affairs, and her study of 25 European Parliament debates (2021) found MEPs using emotions according to their power position. A chatbot drafting for a minister has no position or domain unless the prompt supplies one. My question is whether its output follows those feeling rules or flattens them into one courteous register. As I read the vacancy, emotional cues are the chatbot displaying feeling towards its user: an empathetic GPT-4 agent raised perceived benevolence but not competence (Pralat, Ischen and Voorveld, 2025), and emotional capacity is part of how human-like a chatbot is perceived to be (Ischen, Wang and Smit, 2026).

Emotional appeals invite feeling about the policy issue, which Brader (2005) gives reason to code as discrete emotions, since enthusiasm and fear cues moved voters differently.

Study 1, in year one, would build the corpus from the tasks politicians and civil servants give chatbots, such as a briefing note, a reply to a citizen or a crisis statement, collected with the team and the project's expert meetings and grounded in texts such as parliamentary questions. Prompts would vary by policy domain along the feeling-rule contrast, crisis or routine framing, the prompter's role and ideological framing, since models lean politically and can be steered (Rozado, 2024). Each prompt would go to several models and versions in English and a second EU language, with repeated runs and version and date logged, because in my own system the reference moved with the models. Indicatively, 200 prompts, five models and ten runs in two languages give about 20,000 outputs. I would read a purposive subsample closely and draft the codebook from that reading, piloted by two trained coders, since the cases two coders split on are where a codebook gets written.

Study 2, in year two, would measure emotion with several instruments, since two lexicons can reach opposite conclusions on the same text (Czarnek and Stillwell, 2022). I would start from the eight emotions and validation approach of Widmann and Wich (2023) rather than port their German models, train transformers for both languages on human-coded outputs, and keep NRC as a second lexicon. An LLM annotator would add scale, since zero-shot GPT-4 tracked human-coded sentiment in political speeches better than the JST and rJST topic-sentiment models that Pipal, Schoonvelde, Schumacher and Boiten (2024) tested and packaged for R. A model judging model text is circular, so, as Ziems and colleagues (2024) recommend, it would work inside a human team: trained coders would code about 1,500 outputs, with Krippendorff's alpha reported, sized by adapting the manuscript's simulation. With the manuscript's invariance and differential item functioning methods I would test whether each instrument measures the same thing across models and prompt types and, following the preregistered multiverse design of Pipal, Bakker, Schumacher and van der Velden (2024), report which differences survive every reasonable choice.

Study 3, in year three, would be interpretive, a study in its own right rather than a pilot for the counting. Using Emotion Discourse Analysis as in Sanchez Salgado and Bos (2024), and Koschut's (2018) argument that emotion words can reinforce or change hierarchies, I would ask whom an output casts as victim, threat or authority and which policy options its emotional language makes legitimate, next to European Parliament speeches on the same issues. Cases where instruments and coders disagree would return to this reading as findings. I have not been trained in these methods, so I would learn them under Dr Sanchez Salgado rather than claim them. Each study would become an article, some co-authored with the team, with year four for the dissertation. I would reshape all of this with you.

The PhD would give my work a theory of what emotion does in political text and methods to read it, not only count it; I would bring pipelines that code multilingual text at scale and, having been misled by my own figures, the habit of checking a number before it reaches a policymaker.

My master's is in data analytics rather than one of the listed social sciences; computational social science is the nearest match. I have not run behavioural experiments or controlled prompt experiments on LLMs: my quantitative work is observational and simulation-based. I have no formal training in political psychology or the political sociology of emotions, and my dissertation's human-coded validation covered sentiment, not emotion. I have no publications; the manuscript with Dr Haig is complete but not yet submitted. I have not taught at university level, and I do not speak Dutch yet but would begin learning it on arrival.

I would relocate to within commuting distance of Amsterdam, can start on 1 January 2027, and would take on the teaching and project work and join the Hot Politics Lab seminars. My writing sample, the methodology chapter of my sole-authored MSc dissertation, follows in this file. Thank you for considering my application. I would welcome the chance to discuss SP1 with you.

Yours sincerely,

Asad Ikram

Chevening Scholar 2024/25 · MSc Data Analytics (Distinction), University of Portsmouth

Data Engineer, M+C Saatchi Fluency · Founder & CTO, ArtemisAI

asad.ikram53@gmail.com · +44 7482 376417

asad-ikram-mc.github.io/portfolio · linkedin.com/in/asad-ikram98

Writing sample

Asad Ikram · UvA vacancy 15455, A.I. Feels SP1

What follows: Chapter 6 (Methodology) of my MSc dissertation, *Cross-Platform Sentiment Analysis of Public Reaction to UK Economic Policies*, University of Portsmouth, September 2025, supervised by Dr Ella Haig (project mark 78/100; MSc awarded with Distinction).

Authorship: I am the sole author of the dissertation and of every page in this excerpt.

Pages: the title page and pages 26 to 34 of the original. Page 31, a screenshot of code, is omitted to keep the upload under the 4 MB limit. Section 6.4 describes the emotion analysis with the NRC lexicon. The full dissertation is available on request.

A correction I made later: re-auditing this work for a methods manuscript with Dr Haig, I found that the BERT macro-F1 of 0.878 reported in the dissertation was scored against the lexicon-seeded training labels rather than the 200-post human-coded set. The manuscript states this.



MSc Data Analytics

Final Year Project

Cross-Platform Sentiment Analysis of Public Reaction to UK Economic Policies

By

Asad Ikram (UP2291687)

September 2025

Project Unit: U22244 Masters

Project Supervisor: Dr Ella Haig

Computing and Mathematics Program Area

Chapter 6. Methodology

This chapter detailed the design of the study, and the technical methodology employed to answer the research questions. It described the overall research design (including how variables were defined and controlled), the data preprocessing pipeline, the machine learning models and their selection rationale, the sentiment and emotion analysis approach, the time-series forecasting method, and the evaluation plan (including metrics and statistical tests). Implementation specifics such as software architecture, hardware used, and steps taken for reproducibility were also included. The methodology was developed with the dual aim of rigorously addressing the research questions (RQs) and aligning with the constraints and goals outlined in Section 4.

6.1 Research Design and Rationale

Key study followed a mixed-methods quantitative approach, primarily leveraging computational text analysis for sentiment classification and statistical time-series analysis for forecasting.

The research problem was framed as follows:

- **Independent variables:** platform (Twitter, Reddit, YouTube, Quora), policy topic (categorical variable indicating which economic policy area a post referred to).
- **Dependent variable:** sentiment expressed in a post (ordinal variable labelled as -1 = negative, 0 = neutral, $+1$ = positive).
- **Latent variables of interest:** emotion categories present in text (anger, fear, trust, joy, etc., per lexicons) used to interpret sentiment trends rather than as direct model outputs.

Four working hypotheses guided the RQs:

- **RQ1:** transformer-based models (BERT) would significantly outperform simpler baselines in capturing sentiment nuances in this policy context.
- **RQ2:** sentiment trend directions would often be similar across platforms during shared events, but magnitudes and reaction times would differ (e.g., Twitter reacting faster and more negatively than YouTube).
- **RQ3:** sentiment time series would exhibit autocorrelation, enabling short-term forecasting barring major disruptive events.
- **RQ4:** variations would emerge by policy topic and over time e.g., “tax increases” skewing negative, “economic stimulus” yielding mixed sentiment.

A true experimental design was not possible; the study instead conducted observational analysis of naturally occurring social-media data. For model evaluation, a temporal train–test split was implemented: all labelled data up to a fixed cut-off date formed the training set, while subsequent data formed the test set. Within the training set, an 80/20 stratified split ensured balanced class representation for validation and hyperparameter tuning. This structure simulated real-world policy-monitoring scenarios training on past data, then applying models to unseen, future data and avoided information leakage (e.g., posts by the same users appearing in both train and test).

Two key assumptions underpinned this design:

- a. Posts labelled as positive, neutral, or negative genuinely reflected the author's stance toward the policy in question (despite possible noise from sarcasm or ambiguous phrasing).
- b. The combined coverage of Twitter, Reddit, YouTube, and Quora provided a sufficiently broad view of UK public opinion to justify aggregated analyses despite each platform's demographic bias.

6.2 Data Preprocessing Pipeline

The preprocessing logic are with the following main stages:

1. Data Ingestion and Schema Alignment

- 1.1 Data were collected via platform APIs and saved in a unified schema: platform, comment_id, parent_post_id, timestamp, text etc.
- 1.2 Duplicate entries were removed, including retweets/reposts and identical cross-posted content.

- Basically, used special tools api to pull posts into one neat table with the same columns for all platforms, then deleted repeated posts.

2. Text Cleaning and Normalization

- 2.1 URLs were replaced with [URL].
- 2.2 User mentions (e.g., @username) were removed.
- 2.3 Hashtags were kept with the # stripped so their topical value remained.
- 2.4 HTML entities and markup were removed; emojis were converted to textual descriptions using the emoji Python library.
- 2.5 Text was lowercased, with optional auxiliary storage of the original casing.
- 2.6 Common contractions were expanded ("don't" → "do not").
- 2.7 Platform-specific noise was removed (e.g., RT prefixes on Twitter, > blockquote markers on Reddit).
- 2.8 For punctuation, emphasis marks like ? and ! were preserved, while non-sentiment-bearing punctuation was stripped.

- Basically, cleaned the text so it's tidy and consistent. For example, replaced web links with "[URL]", turned emojis into words, removed extra platform junk, and made everything lowercase for easier processing.

3. Tokenization And Sequence Handling

- 3.1 BERT: Used HuggingFace's bert-base-uncased WordPiece tokenizer, with sequences padded/truncated to a max length of 128 tokens.

3.2 BiLSTM: Tokenized via NLTK/spaCy; vocabulary of the top ~20,000 words; out-of-vocabulary tokens replaced with UNK (unknown token); sequences padded/truncated to 128 tokens. Lemmatization was applied for the LSTM pipeline.

- Basically, tokenization means breaking text into smaller pieces (words or subwords) and turning them into numbers for the model. We used different tokenizers for BERT and LSTM, kept posts the same length (128 tokens), and replaced words not in the vocabulary with a special “UNK” token for “unknown.”

4. Language Filtering

4.1 Non-English posts were detected with langdetect and dropped; ~98% of data remained English.

- Basically, used a language detection tool to find and remove posts not written in English, so almost all remaining posts are in English.

5. Special Handling

5.1 Negations in LSTM input were marked via bigram joins (not good → NOT_good).

5.2 Laughter/onomatopoeia were preserved.

5.3 Repeated punctuation was collapsed but flagged with a EMPH (emphasis token) token.

- Basically, gave the model extra clues: joined “not” to the word after it so it knows the meaning flips, kept laughter like “haha” because it shows feeling, and used an EMPH token (short for “emphasis”) to show when punctuation meant something was extra strong.

6. Labelling

6.1 VADER’s compound sentiment score assigned initial labels:

6.1.1 $\geq 0.05 \rightarrow$ positive

6.1.2 $\leq -0.05 \rightarrow$ negative

6.1.3 otherwise $\rightarrow$ neutral

6.2 A manually annotated set of 200 posts (balanced across platforms) served as a high-quality validation/test set. Inter-annotator agreement (Cohen’s $\kappa \approx 0.78$) confirmed reliability. This means your annotators agreed much more than would be expected by chance, and manually annotated set is considered reliable.

- Basically, used a program called VADER to tag posts as positive, negative, or neutral. Then checked a smaller set by hand. Their agreement score (Cohen’s kappa) shows how often they agreed after accounting for lucky guesses (0.78), means strong agreement. Finally, stored each post in the right number format for both models.

Each post was stored in both token sequence format (for LSTM) and BERT input-ID + attention mask format. Fig 6.2 shows part of preprocessing code as example to demonstrate above steps that I took.

Fig 6.2 Preprocessing Code Snippet

```

116 # ----- Regex & Helpers -----
117 CLEAN_URLS = re.compile(pattern="http[s|www|https|s|https|s]*", re.IGNORECASE)
118 CLEAN_HTML = re.compile(pattern="<.*>")
119 CLEAN_HANDLES = re.compile(pattern="(@[a-zA-Z0-9_]+)")
120 CLEAN_RT = re.compile(pattern="(\bRT\b)")
121
122 # Light cleaning for VADER (preserve punctuation, emojis, emoticons)
123 def clean_for_vader(text):
124     t = text or ""
125     for patt in (CLEAN_URLS, CLEAN_HTML, CLEAN_HANDLES, CLEAN_RT):
126         t = patt.sub(repl="", t)
127     return t.strip()
128
129 # Full cleaning for tokenization & lemmatization
130 CLEAN_NDN = re.compile(pattern="[A-Za-z0-9_]+")
131 MD_LINKS = re.compile(pattern="(.+?)\((.+?)\)")
132 AST_UNDER = re.compile(pattern="([a-zA-Z0-9_]+)")
133 YT_TS = re.compile(pattern="([a-zA-Z0-9_]+)")
134 QUORA_QA = re.compile(pattern="([a-zA-Z0-9_]+)\(Question|Answer\):", flags=re.IGNORECASE)
135
136 def clean_text(text):
137     t = text or ""
138     for patt in (CLEAN_URLS, CLEAN_HTML, CLEAN_HANDLES, CLEAN_RT):
139         t = patt.sub(repl="", t)
140     return CLEAN_NDN.sub(repl="", t).strip()
141
142 def platform_specific(row):
143     p, t = row.platform, row.clean_text
144     if p == "Twitter":
145         t = re.sub(pattern="(\bRT\b)", repl="", t)
146     elif p == "Reddit":
147         t = MD_LINKS.sub(repl="", t)
148     elif p == "AST_UNDER":
149         t = AST_UNDER.sub(repl="", t)
150     elif p == "YouTube":
151         t = YT_TS.sub(repl="", t)
152     elif p == "Quora":
153         t = QUORA_QA.sub(repl="", t)
154
155
156 # ----- Core Preprocessing -----
157 def preprocess_comments(df, nlp, stop_words, bert_tok, glove_dict, vader, sarcasm_detector):
158     log.info(f"--- Preprocessing comments ({len(df):} rows) ---")
159     # preserve original
160     df["original_text"] = df["text"]
161     df["timestamp"] = pd.to_datetime(df["timestamp"], errors="coerce")
162
163     # 1) Light cleaning + VADER
164     df["vader_text"] = df["original_text"].progress_apply(clean_for_vader)
165     df["vader_compound"] = df["vader_text"].progress_apply(lambda x: vader.polarity_scores(x)["compound"])
166     df["vader_label"] = df["vader_compound"].progress_apply(vader_label)
167
168     # 2) Full cleaning + lemmatization
169     df["clean_text"] = df["original_text"].progress_apply(clean_text)
170     df["clean_text"] = df.progress_apply(platform_specific, axis=1)
171     def lemmatize(tx):
172         doc = nlp(tx.lower())
173         return " ".join(
174             tok.lemma_ for tok in doc
175             if tok.is_alpha and tok.text not in stop_words and len(tok) > 2
176         )
177     df["processed_text"] = df["clean_text"].progress_apply(lemmatize)
178
179     # 3) Token lists for downstream models
180     df["bert_tokens"] = df["processed_text"].progress_apply(
181         lambda x: bert_tok(x, add_special_tokens=True, truncation=True, max_length=512).input_ids
182     )
183     df["glove_tokens"] = df["processed_text"].progress_apply(
184         lambda x: [w for w in x.split() if glove_dict is not None and w in glove_dict]
185     )
186
187     # 4) Distilled sarcasm classifier
188     def sarcasm_score(text):
189         out = sarcasm_detector(text[:256])[0]
190         return (out["score"] * 100) if out["label"].upper().startswith("SARCASTIC") else 0.0
191     df["sarcasm_pct"] = df["original_text"].progress_apply(sarcasm_score)
192

```

6.3 Sentiment Classification Models

Two supervised classifiers were implemented, and details are as follows:

- **BERT (bert-base-uncased)**
 - Fine-tuned with a classification head on the [CLS] token.
 - Hyperparameters: learning rate = 2e-5, batch size = 32, epochs = 3, class weighting applied.
 - Trained on GPU (Nvidia T4, Colab Pro +)
- Basically, BERT is a powerful pre-trained language model. Added a small classifier on top of its special CLS token (a summary token) and trained it with careful settings (learning speed, data chunk size, and number of rounds) on a fast graphics card.
- **BiLSTM**
 - Embedding layer with pre-trained **GloVe 100d** vectors, initially frozen then fine-tuned.

- Bidirectional LSTM with 64 hidden units per direction; dropout = 0.3; dense ReLU layer + softmax output.
- Hyperparameters: learning rate = 1e-3, batch size = 64, epochs up to early-stopping patience = 3.
- Basically, BiLSTM is a type of recurrent network that reads text forward and backward to understand context. It started with word meanings from **GloVe embeddings** (pre-learned word maps), used 64 hidden “memory cells” in each direction, and had layers to avoid overfitting (**dropout**) and to classify the text with softmax (turns outputs into probabilities).

Calibration: Both models were post-processed with **temperature scaling** on validation data. BERT’s Expected Calibration Error (ECE) dropped from ~0.08 to ~0.05 after scaling; BiLSTM’s from ~0.12 to ~0.07. Brier scores were also computed.

- Basically, after training I adjusted the confidence of the models with temperature scaling so that their predicted probabilities better matched reality. This made BERT’s confidence more reliable (ECE (it tells us if the model’s confidence can be trusted) went down from 0.08 to 0.05) and improved BiLSTM as well. The Brier score (it shows how close the model’s probability predictions are to the truth, smaller means more accurate and reliable) checked overall accuracy of probabilities.

6.4 Emotion Analysis

The NRC Emotion Lexicon was applied to the preprocessed text. For each post, counts of words in the eight basic emotion categories and two sentiment categories were calculated. Proportional emotion scores were aggregated over time and compared with predicted sentiment classes.

In short used a word–emotion dictionary to see which feelings (like anger, joy, fear) appeared in each post, tracked how they changed over time, and compared them with the model’s sentiment results.

6.5 Time-Series Forecasting

Key Sentiment forecasts used Facebook Prophet. The target was the daily sentiment index:

Index = (number of positive posts) – (number of negative posts)

A 3-day moving average smoothed the series before modelling. Prophet configuration: yearly + weekly seasonality, custom regressors for major economic policy dates, changepoint detection enabled. Training data extended to May 2025; June 2025 served as the forecast period.

Evaluation: MAE, MAPE, and coverage of 95% prediction intervals ($\approx$ 94% coverage achieved).

Conclusively built a forecasting model with Prophet to predict daily sentiment, smoothed the data with a 3-day average, included key event dates, and tested accuracy with standard error measures. Fig 6.5 shows part of ml code as example to demonstrate above steps that I took.

6.6 Evaluation Plan

Evaluation was designed to systematically address each Research Question (RQ) with both quantitative performance metrics and statistical validation. The plan covered four dimensions: model performance (RQ1), sentiment classification accuracy (RQ2), forecasting accuracy (RQ3), and granular variations (RQ4).

RQ1 – Model Effectiveness

- **Classification Metrics**

The supervised models (BERT and BiLSTM) were evaluated on the held-out test set using Accuracy, Precision, Recall, and F1-score (macro-averaged and per-class). Macro-F1 was prioritized because it treats each sentiment class (positive, neutral, negative) equally, avoiding dominance of the neutral class.

- **Baselines**

Results were compared against (a) VADER (rule-based lexicon classifier) and (b) a simple logistic regression model using bag-of-words features. This established whether advanced models truly added value over traditional methods.

- **Error Analysis**

Confusion matrices were inspected to understand common misclassifications, e.g., neutral vs. negative confusions.

- **Calibration**

Expected Calibration Error (ECE) and Brier Score were computed to assess whether predicted probabilities aligned with true likelihoods (important for forecasting). Temperature scaling was applied post-training.

- **Statistical Tests**

To check significance of performance differences between BERT and BiLSTM, McNemar's test was performed on paired predictions. Bootstrap resampling (1,000 iterations) provided 95% confidence intervals for key metrics (Accuracy, F1), ensuring robustness.

RQ2 – Sentiment Classification Accuracy

- **Classification Metrics**

The models will be tested on a held-out set and we will report Accuracy and **macro-F1** (our main score). We will also show Precision, Recall, and F1 for each class (Negative, Neutral, Positive), the class counts, and a confusion matrix.

- **Baselines**

Results will be compared against three simple baselines: a majority-class guesser, VADER (rule-based), and logistic regression with bag-of-words. A single summary table will make it clear how BiLSTM and BERT improve over these baselines.

- **Error Analysis**

Will inspect the confusion matrix to see where mistakes happen most, especially **Neutral** ↔ **Negative** and **Positive** ↔ **Neutral** mix-ups. Will also list a few typical misclassified examples and check errors by **policy category** to spot patterns.

- **Calibration**

Will measure **Brier score** and **Expected Calibration Error (ECE)** to see if predicted probabilities are trustworthy. Reliability plots will be shown **before and after temperature scaling** (fitted on the validation set), and we will report the improved Brier/ECE after scaling.

- **Statistical Tests**

To check if BERT truly beats BiLSTM (and baselines), we will run **McNemar's test** on a 2×2 table of paired outcomes (correct/incorrect per item for the two models) and report the χ^2 statistic and **p-value** ($\alpha = 0.05$). To show uncertainty in our scores, will use **bootstrap resampling** (1,000 draws of the test set with replacement) to compute **95% confidence intervals** for **Accuracy** and **macro-F1** for each model. For the difference in macro-F1 between models, we will report the **bootstrap CI for the difference** (and consider a simple **permutation test** as a robustness check). Will avoid t-tests on F1 because it is not normally distributed; all tests are **paired** to respect that models are evaluated on the **same** items.

RQ3 – Predictive Capability

- **Forecasting Metrics**

Prophet forecasts were evaluated on unseen June 2025 data. Accuracy was measured using Mean Absolute Error (MAE) and Mean Absolute Percentage Error (MAPE).

- **Event Sensitivity**

Forecast directionality was checked i.e., did the model correctly predict upward or downward sentiment shifts around key economic policy announcements?

- **Uncertainty Calibration**

Coverage of 95% prediction intervals was measured (target $\approx 90\text{--}95\%$). High coverage meant Prophet's uncertainty bands realistically reflected volatility in sentiment trends.

RQ4 – Granular Variations

- **By Policy Topic**

Sentiment distributions were aggregated for each policy category (e.g., Taxation, Welfare, Inflation). Chi-square tests assessed whether distributions differed significantly by topic.

▪ **Temporal Shifts**

Sentiment was compared across key time slices (e.g., pre-/post-high-inflation periods) to evaluate whether sentiment trajectories reflected external economic conditions.

6.6 Implementation and Reproducibility

- Key Languages/Libraries: Python 3.11; pandas, spaCy, NLTK, scikit-learn, PyTorch, HuggingFace Transformers, TensorFlow/Keras, Prophet, matplotlib/Plotly.
- Random Seeds: Fixed (42) for reproducibility except in Prophet.
- Hardware: Colab GPU (Nvidia T4, ~ 80 GB).
- Pipeline Modularity: Data collection → preprocessing → model training/evaluation → forecasting.
- Security: API credentials stored securely, not in notebooks.

This design integrated lexicon bootstrapping, deep learning classification, lexicon-based emotion analysis, and statistical forecasting ensured comprehensive, reproducible analysis aligned with the study's RQs.

From an IT design perspective, the implementation emphasized modularity. Each stage (data collection, preprocessing, modeling, forecasting, and visualization) was encapsulated, allowing components to be updated independently. For example, the sentiment classification model could be replaced (e.g., swapping BERT for a newer transformer) without altering the upstream data collection pipeline. Configuration elements such as keyword lists, stopword sets, and preprocessing rules were moved into external files to facilitate straightforward adjustments.

In conclusion, the methodological design combining lexicon-based labeling with supervised machine learning and integrating classification with forecasting was deliberately comprehensive to address the research questions from multiple angles. By balancing the use of state-of-the-art transformer models with interpretable lexicons and robust statistical validation, the project achieved both accuracy and explainability. The subsequent chapters (Section 7 and Section 8) describe the application of this methodology to the collected data and present the resulting insights before evaluating the project outcomes against the stated goals.